

Fundamentos de motores de videojuegos con C++



Facultad de
Ciencias
UNAM

Facultad de Ciencias, UNAM

Práctica 02

Vectores 2D, Entrada Continua y Movimiento

En esta práctica implementamos los primeros cimientos matemáticos y arquitectónicos de nuestro motor de videojuegos: una estructura vectorial en dos dimensiones (`Vector2`) con soporte para suma, resta, producto escalar, cálculo de magnitud y normalización. Conectamos la lectura continua del teclado mediante `SDL_GetKeyboardState` bajo la arquitectura de *callbacks* de SDL3 para mover un objeto en pantalla con *delta time* (Δt). Descubrimos y resolvemos el famoso "error de movimiento diagonal", aprendemos a modularizar código en archivos de cabecera (`.hpp`), introducimos el principio de *composición sobre herencia* mediante entidades simples, abstraemos la compilación multiplataforma con `CMake` y realizamos nuestra primera entrega gestionada a través de un repositorio en GitHub.

Objetivo

Implementar una estructura matemática `Vector2` en C++ con operaciones de magnitud y normalización, capturar entrada continua de teclado con `SDL_GetKeyboardState` bajo el sistema de callbacks de SDL3 para desplazar un objeto en pantalla de manera uniforme con Δt , modularizar el código extrayendo la estructura a un archivo de cabecera propio (`.hpp`), organizar el estado mediante composición (`struct Character`), compilar el proyecto de forma multiplataforma utilizando `CMake` y publicar el resultado en un repositorio de GitHub.

Objetivos secundarios

- Comprender la relación entre las fases del *game loop* y la API de callbacks de SDL3 (`SDL_AppInit`, `SDL_AppIterate`, `SDL_AppEvent`, `SDL_AppQuit`).

- Distinguir entre el manejo de eventos discretos (`SDL_AppEvent`) y la consulta de entrada continua en tiempo real (`SDL_GetKeyboardState`).
 - Entender qué es un vector 2D en videojuegos tanto geométrica como algebraicamente.
 - Implementar y sobrecargar operadores para operaciones vectoriales elementales (suma, resta, escala).
 - Calcular la magnitud euclidiana de un vector y entender la utilidad computacional de la magnitud al cuadrado.
 - Comprender y corregir el error del movimiento diagonal mediante la normalización vectorial.
 - Entender qué son los archivos de cabecera en C++, para qué sirven y por qué se prefiere la extensión `.hpp` sobre `.h` en C++ moderno.
 - Aplicar el principio de diseño de *composición sobre herencia* al construir entidades del juego.
 - Comprender el propósito de un generador de compilación (`CMake`) frente a la invocación manual del compilador.
 - Conocer la diferencia entre el Δt variable de renderizado y el Δt fijo de físicas (*fixed timestep*).
 - Vincular un repositorio local a un servidor remoto en GitHub y realizar `push` de la rama principal.
 - (*Avanzado*) Implementar confinamiento en pantalla (*clamping*) o persecución de coordenadas hacia el cursor del mouse.
-

Antecedentes

1. La API de callbacks de SDL3 y el Game Loop

En la práctica anterior estudiamos el *game loop* clásico en un único bloque `main()` con un ciclo `while (is_running)`. En esta práctica partimos de la arquitectura unificada que introduce SDL3: el sistema de **callbacks multiplataforma** (`#define SDL_MAIN_USE_CALLBACKS 1`).

En este modelo, nosotros no escribimos el `while`: le delegamos el control del bucle al sistema operativo y a SDL, y nosotros implementamos cuatro funciones específicas que SDL invocará en el momento adecuado:

Fase del Game Loop	Callback de SDL3	Propósito en el Motor
Inicialización	<code>SDL_AppInit</code>	Se ejecuta una sola vez al arrancar. Inicializa subsistemas, crea la ventana y el <i>renderer</i> , y reserva la memoria del estado global.
Eventos	<code>SDL_AppEvent</code>	Se invoca por cada evento individual en la cola (cerrar ventana, redimensionar, clic de mouse).
Actualización y Dibujo	<code>SDL_AppIterate</code>	Es el corazón del ciclo. Se llama repetidamente en cada <i>frame</i> para calcular la física, procesar entrada continua y dibujar en pantalla.
Limpieza	<code>SDL_AppQuit</code>	Se ejecuta al salir. Libera texturas, destruye el <i>renderer</i> y la ventana, y apaga SDL.

Cada función retorna un código de estado (`SDL_AppResult`):

- `SDL_APP_CONTINUE` : el programa sigue corriendo normalmente.
- `SDL_APP_SUCCESS` : el programa debe terminar limpiamente sin errores.
- `SDL_APP_FAILURE` : ocurrió un error crítico y el programa debe abortar.

2. Delta Time en Callbacks y Timestep Fijo de Físicas

Cuando usamos callbacks, la plataforma decide con qué frecuencia llama a `SDL_AppIterate` . Podemos sugerir una tasa objetivo con `SDL_SetHint(SDL_HINT_MAIN_CALLBACK_RATE, "60")` , pero la tasa real dependerá del monitor, la carga del procesador y el sistema operativo.

Para lograr que el movimiento ocurra a la misma velocidad en cualquier computadora (sea a 30, 60 o 144 FPS), debemos medir el tiempo exacto transcurrido entre llamadas consecutivas a `SDL_AppIterate` :

$$\Delta t = \frac{t_{\text{actual}} - t_{\text{anterior}}}{1000.0} \quad (\text{en segundos})$$

Si queremos que un objeto avance a una velocidad de 300 px/s, la distancia a recorrer en este *frame* específico es:

$$\text{desplazamiento} = \vec{v} \cdot (\text{velocidad} \cdot \Delta t)$$

Nota

Delta time variable vs. Timestep fijo (*Fixed Timestep*):

Usar Δt variable es ideal para renderizado fluido y movimiento continuo simple. Sin

embargo, en simulaciones físicas complejas (colisiones, cálculo de gravedad o plataformas), variaciones abruptas de Δt (por ejemplo, si el sistema operativo se congela 100 ms) pueden provocar que un proyectil atravesase una pared (*tunneling*) o que una parábola de salto varíe demasiado (y tu personaje salga volando).

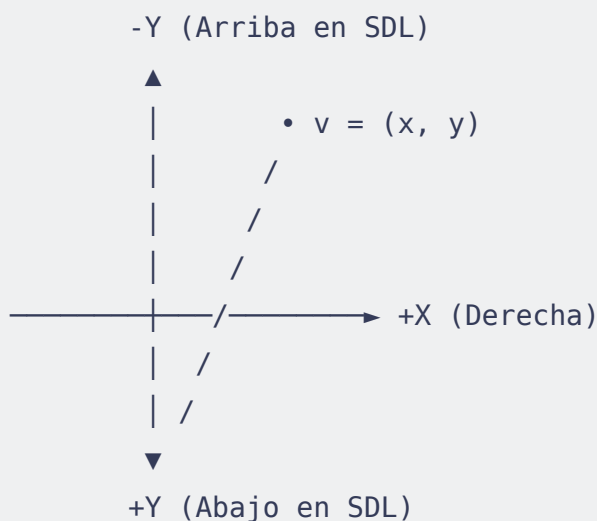
Por eso, los motores profesionales separan la actualización en dos niveles:

1. **Update de renderizado:** corre con Δt variable cada vez que el monitor puede dibujar.
2. **Physics Update:** corre a pasos de tiempo estrictamente fijos y constantes (por ejemplo, exactamente a $1/60 \approx 0.0166$ s por paso), utilizando un acumulador de tiempo.

3. Vectores 2D en Videojuegos

Un vector en dos dimensiones es un par ordenado $\vec{v} = (x, y)$ que puede representar dos conceptos geométricos fundamentales:

1. **Una posición:** un punto en el espacio de la pantalla respecto al origen $(0, 0)$.
2. **Un desplazamiento o dirección:** hacia dónde y con qué intensidad apunta una fuerza o movimiento.



⚠ Mucho ojito

El sistema de coordenadas de pantalla:

Recuerda que en computación gráfica 2D (incluyendo SDL), el eje X crece hacia la **derecha**, pero el eje Y crece hacia **abajo**. El punto $(0, 0)$ es la esquina superior izquierda de la ventana. Por lo tanto:

- Moverse hacia **arriba** significa restar en Y (ej. $y = -1$).
- Moverse hacia **abajo** significa sumar en Y (ej. $y = +1$).

Operaciones vectoriales elementales

1. **Suma de vectores:** Permite aplicar un desplazamiento a una posición actual:

$$\vec{a} + \vec{b} = (a_x + b_x, a_y + b_y)$$

2. **Resta de vectores:** Permite obtener la distancia y dirección desde un punto $\vec{a}$ hacia un punto $\vec{b}$:

$$\vec{d} = \vec{b} - \vec{a} = (b_x - a_x, b_y - a_y)$$

3. **Multipliación por escalar:** Permite cambiar la escala o magnitud de un vector sin alterar su dirección:

$$\vec{v} \cdot s = (x \cdot s, y \cdot s)$$

4. **Magnitud (Longitud euclidiana):** Por el Teorema de Pitágoras, la longitud de la flecha es:

$$\|\vec{v}\| = \sqrt{x^2 + y^2}$$

5. **Magnitud al cuadrado:**

$$\|\vec{v}\|^2 = x^2 + y^2$$

Calcular una raíz cuadrada (`std::sqrt`) es una operación computacionalmente costosa. Cuando solo queremos saber si un vector es cero o comparar qué objeto está más cerca, comparamos las magnitudes al cuadrado ($\|\vec{a}\|^2 < \|\vec{b}\|^2$), ahorrando ciclos de CPU valiosos en cada *frame*.

🔗 **0x5F3759DF**

Hay un dato curioso relacionado con el párrafo anterior, investiga este número.

4. El error del movimiento diagonal y la normalización

Cuando un jugador presiona un botón de dirección (normalmente las flechas o WASD; joysticks o cruceta en los controles de consola), el vector resultante tiene longitud 1:

- Derecha ('D'): $\vec{v} = (1, 0) \implies \|\vec{v}\| = \sqrt{1^2 + 0^2} = 1$
- Abajo ('S'): $\vec{v} = (0, 1) \implies \|\vec{v}\| = \sqrt{0^2 + 1^2} = 1$

Pero cuando el jugador presiona dos teclas al mismo tiempo (ej. 'W' y 'D' para avanzar en diagonal hacia arriba y a la derecha), el vector de entrada compuesto es $\vec{v} = (1, -1)$. Calculemos su longitud:

$$\|\vec{v}_{\text{diag}}\| = \sqrt{1^2 + (-1)^2} = \sqrt{1 + 1} = \sqrt{2} \approx 1.4142$$

¡El vector diagonal es un 41.4% más largo! Si usamos este vector directamente para mover al personaje, se desplazará a $1.414 \times 300 = 424.26$ px/s en diagonal, otorgándole una ventaja de velocidad injusta. Este fue un error clásico en videojuegos históricos como *Doom*, *Quake* o *GoldenEye 007*, donde los jugadores veteranos descubrieron que moverse de lado en diagonal (*strafe-running*) permitía recorrer mapas mucho más rápido.

Para corregirlo, debemos **normalizar** el vector. Un vector normalizado (o vector unitario $\hat{v}$) conserva exactamente la misma dirección, pero su longitud se ajusta para que sea exactamente 1:

$$\hat{v} = \frac{\vec{v}}{\|\vec{v}\|} = \left(\frac{x}{\|\vec{v}\|}, \frac{y}{\|\vec{v}\|} \right) \quad (\text{si } \|\vec{v}\| > 0)$$

Aplicado a nuestro vector diagonal $(1, -1)$:

$$\hat{v}_{\text{diag}} = \left(\frac{1}{\sqrt{2}}, \frac{-1}{\sqrt{2}} \right) \approx (0.7071, -0.7071)$$

Comprobamos su nueva magnitud:

$$\|\hat{v}_{\text{diag}}\| = \sqrt{0.7071^2 + (-0.7071)^2} = \sqrt{0.5 + 0.5} = \sqrt{1} = 1$$

Ahora, sin importar si el jugador se mueve en línea recta o en diagonal, la velocidad siempre será de 300 px/s.

5. Entrada continua vs. Eventos discretos

En SDL existen dos maneras de leer el teclado:

1. Eventos discretos (`SDL_AppEvent`):

Notifica una acción puntual en el instante exacto en que sucede

(`SDL_EVENT_KEY_DOWN` , `SDL_EVENT_KEY_UP`). Adecuada para acciones de una sola vez: saltar, abrir un menú, pausar o disparar un arma semiautomática.

2. Consulta de estado continuo (`SDL_GetKeyboardState`):

Devuelve un apuntador a un arreglo de booleanos con el estado en tiempo real de todas las teclas en este preciso *frame*. En la documentación puedes encontrar funciones relacionadas, pero esta es la herramienta adecuada para movimiento continuo: queremos saber si la tecla 'W' *sigue presionada ahora mismo* para continuar desplazando al personaje.

6. Organización de código: Archivos de cabecera (`.hpp` vs `.h`)

A medida que nuestro código crece, mantener todo dentro de `main.cpp` se vuelve insostenible. En C++, dividimos el software en dos tipos de archivos:

- **Archivos fuente (`.cpp`):** contienen la lógica ejecutable y las definiciones de funciones. Se compilan de forma independiente para generar archivos objeto (`.o`).
- **Archivos de cabecera (headers):** contienen declaraciones de tipos, estructuras, clases y firmas de funciones que otros archivos necesitan conocer para interactuar con ellas.

¿Por qué usamos `.hpp` en lugar de `.h` ?

Históricamente, el lenguaje C utiliza la extensión `.h`. En C++ moderno, la convención estándar es nombrar los encabezados como `.hpp` (o `.hxx`) para indicar explícitamente que el archivo contiene código propio de C++ (plantillas, espacios de nombres, clases, sobrecarga de operadores y `constexpr`). Esto ayuda a los editores de código, herramientas de análisis estático y compiladores a aplicar de inmediato las reglas de C++.

Para evitar que un archivo de cabecera sea procesado múltiples veces en una misma compilación (lo que causaría errores de redefinición), colocamos en la primera línea la directiva de preprocesador:

```
#pragma once
```

⚠ Mucho ojo

De hecho esto último es algo controversial, si te interesa investiga la diferencia entre `#pragma once` y los bloques de guardas (`#ifndef`, `#define`, `#endif`).

7. Composición sobre herencia (*Composition over Inheritance*)

Un principio fundamental en el diseño de software para videojuegos es: "**Prefiere composición sobre herencia**" (*favor composition over inheritance*).

- En un enfoque rígido de herencia, podríamos vernos tentados a decir: "*un Personaje ES un Vector2*" (`class Character : public Vector2`). Pero esto no tiene sentido conceptual: un personaje no es un punto matemático; un personaje **TIENE** una posición espacial (`Character has-a Vector2 position`).
- En esta práctica crearemos una entidad sencilla `struct Character` que encapsula como miembros sus atributos espaciales (`Vector2 position`), su tamaño (`Vector2 size`), su velocidad y su color. En la Semana 03 formalizaremos esto en la arquitectura de componentes de nuestro motor.

¿Por qué crear nuestro propio `Vector2` si SDL ya tiene `SDL_FPoint` y `SDL_FRect` ?

SDL3 incluye tipos como `SDL_FPoint` (con miembros `float x, y`) y `SDL_FRect`. Sin embargo, son estructuras básicas de lenguaje C: no poseen constructores, ni operadores sobrecargados (`+`, `-`, `*`), ni métodos para calcular distancias o normalizar vectores.

Además, un motor de videojuegos debe desacoplar sus reglas lógicas y matemáticas de la biblioteca gráfica subyacente. Nuestro motor calcula todo con su propio `Vector2`; al momento de dibujar, simplemente pasamos las coordenadas de nuestro vector a las estructuras que el API de SDL solicita. Realmente es una de las piezas más repetidas entre librerías, así que si lo demás no te convence podríamos dejarlo en "razones didácticas".

8. ¿Qué es CMake y por qué lo usamos a partir de hoy?

En las semanas 00 y 01 compilamos escribiendo comandos directos en la consola (`g++ src/main.cpp -o build/main.out -lSDL3`). Este método funciona para un archivo individual, pero se vuelve frágil cuando el proyecto incluye múltiples archivos de cabecera, subcarpetas y queremos que funcione tanto en Linux como en Windows (MSYS2 o Visual Studio) y macOS.

CMake no es un compilador: es un **generador de sistemas de construcción** (*build system generator*). A partir de un archivo declarativo llamado `CMakeLists.txt`, CMake inspecciona el sistema operativo, localiza dónde está instalada la biblioteca SDL3 y genera los archivos de construcción nativos de cada plataforma (Makefiles en Linux/macOS, soluciones de Visual Studio o Ninja en Windows).

El flujo de trabajo universal con CMake consta de dos pasos en la terminal:

```
# 1. Configurar y generar la carpeta de construcción (solo se hace la
primera vez o si cambias CMakeLists.txt)
cmake -B build
```

```
# 2. Compilar el ejecutable
cmake --build build
```

VS Code Extension

Si usas VS Code, la extensión de CMake automáticamente detecta el archivo de CMake. Te deja elegir contra que compilador, y ejecuta el paso de configuración. Sigue siendo importante que sepas que esta pasando por detras de las cortinas, pero es útil una vez que lo entiendas.

Desarrollo de la práctica

1. La plantilla y compilación con CMake

La carpeta `semana02/plantilla/` contiene la estructura inicial:

```
semana02/plantilla/
├─ CMakeLists.txt
├─ src/
│   └─ main.cpp
```

Revisemos el contenido de `CMakeLists.txt`:

```
cmake_minimum_required(VERSION 3.16)
project(practica02_motor LANGUAGES CXX)

# En prácticas anteriores vimos que podíamos pasar std=c++17
# al compilador. ¿Puedes adivinar que hace la siguiente línea?
set(CMAKE_CXX_STANDARD 20)
set(CMAKE_CXX_STANDARD_REQUIRED ON)

# Buscamos SDL3
find_package(SDL3 QUIET)
```

```

if(NOT SDL3_FOUND)
    find_package(PkgConfig REQUIRED)
    pkg_check_modules(SDL3 REQUIRED IMPORTED_TARGET sdl3)
    set(SDL3_TARGET PkgConfig::SDL3)
else()
    set(SDL3_TARGET SDL3::SDL3)
endif()

# Generamos el ejecutable con el nombre del proyecto
add_executable(${PROJECT_NAME} src/main.cpp)
target_include_directories(${PROJECT_NAME} PRIVATE src)
target_link_libraries(${PROJECT_NAME} PRIVATE ${SDL3_TARGET})

```

Este script configura el estándar C++20, busca SDL3 (de forma nativa o mediante `pkg-config`), añade `src/` como ruta de inclusión de encabezados y vincula la biblioteca.

Compilación inicial

Abre tu terminal dentro de la carpeta `semana02/plantilla/` y ejecuta:

```

# Configuración del proyecto
cmake -B build

# Compilación
cmake --build build

```

Ejecuta el binario generado:

```

# Linux y macOS
./build/practica02_motor

# Windows (MSYS2 ucrt64)
./build/practica02_motor.exe

```

Tip

Si en algún momento necesitas compilar manualmente sin usar CMake, el comando equivalente visto en la Práctica 01 sigue funcionando:

```
g++ -std=c++20 src/main.cpp -o build/practica02_motor -lSDL3
```

Deberás ver una ventana de 960×540 píxeles con fondo gris oscuro y un rectángulo verde centrado. Por ahora, el rectángulo está estático. Cierra la ventana para continuar.

2. Movimiento "a mano" con variables sueltas

En `main.cpp`, observa que dentro de `AppState` tenemos definidas variables sueltas para la posición:

```
float rect_x{440.0f};  
float rect_y{240.0f};  
float rect_width{60.0f};  
float rect_height{60.0f};  
float speed{300.0f};
```

Vamos a mover el rectángulo consultando el estado del teclado en tiempo real. Dirígete a la función `SDL_AppIterate` en `src/main.cpp`.

En la sección **2. Fase de Actualización (Update)**, añade el siguiente código:

```
// 2. Fase de Actualización (Update)  
const bool *keys = SDL_GetKeyboardState(nullptr);  
  
if (keys[SDL_SCANCODE_W] || keys[SDL_SCANCODE_UP])  
{  
    app->rect_y -= app->speed * delta_time;  
}  
if (keys[SDL_SCANCODE_S] || keys[SDL_SCANCODE_DOWN])  
{  
    app->rect_y += app->speed * delta_time;  
}  
if (keys[SDL_SCANCODE_A] || keys[SDL_SCANCODE_LEFT])  
{  
    app->rect_x -= app->speed * delta_time;  
}  
if (keys[SDL_SCANCODE_D] || keys[SDL_SCANCODE_RIGHT])
```

```
{  
    app->rect_x += app->speed * delta_time;  
}
```

Compila y ejecuta:

```
cmake --build build  
./build/practica02_motor
```

Usa las teclas **WASD** o las **flechas de dirección**. Comprueba que el rectángulo ahora se mueve suavemente por la pantalla a una velocidad consistente gracias a `delta_time`.

3. Identificar el *bug* del movimiento diagonal

Con el programa corriendo en el paso anterior:

1. Muévete en línea recta hacia la derecha manteniendo presionada la tecla **D**. Observa la velocidad.
2. Ahora, mantén presionadas **W** y **D** simultáneamente para moverte en diagonal.
3. Nota cómo el rectángulo se desplaza visiblemente más rápido.

Hagamos números: al avanzar simultáneamente en X y en Y , en 1 segundo el objeto se desplaza:

- 300 px en horizontal.
- 300 px en vertical.

La distancia euclidiana total recorrida en un segundo es:

$$\text{Distancia} = \sqrt{300^2 + 300^2} = \sqrt{180000} \approx 424.26 \text{ px}$$

¡El objeto se desplaza a **424.26 px/s** en vez de los 300 px/s configurados!

Si intentáramos corregir esto manteniendo variables sueltas (`rect_x`, `rect_y`), tendríamos que escribir complicadas combinaciones de `if` para detectar cada diagonal y dividir entre $\sqrt{2}$. La solución profesional y elegante es modelar la velocidad mediante un vector 2D.

4. Crear la estructura matemática `Vector2`

Vamos a construir nuestra propia estructura matemática. Para entender cómo funciona antes de modularizarla, primero la declaramos directamente en `src/main.cpp`.

Abre `src/main.cpp` e incluye la librería matemática estándar de C++ `<cmath>` junto con los demás `#include`:

```
#include <cmath>
```

Justo antes de la definición de `struct AppState`, escribe la estructura `Vector2`:

```
struct Vector2
{
    float x{0.0f};
    float y{0.0f};

    // Constructores
    constexpr Vector2() = default;
    constexpr Vector2(float x, float y) : x(x), y(y) {}

    // Sobrecarga de operadores aritméticos
    Vector2 operator+(const Vector2 &other) const
    {
        return {x + other.x, y + other.y};
    }

    Vector2 operator-(const Vector2 &other) const
    {
        return {x - other.x, y - other.y};
    }

    Vector2 operator*(float scalar) const
    {
        return {x * scalar, y * scalar};
    }

    // Métodos de magnitud
    float length_squared() const
    {
```

```

        return x * x + y * y;
    }

    float length() const
    {
        return std::sqrt(length_squared());
    }

    // Normalización: convierte el vector a magnitud 1 conservando la
    // dirección
    Vector2 normalized() const
    {
        float len = length();
        if (len > 0.0001f) // Protección contra división entre cero
        {
            return {x / len, y / len};
        }
        return {0.0f, 0.0f};
    }
};

```

Analicemos sus características clave:

- `constexpr`: permite que vectores constantes se resuelvan en tiempo de compilación.
- `operator+`, `operator-`, `operator*`: sobrecargan los operadores para escribir expresiones matemáticas naturales en C++, por ejemplo `pos = pos + (dir * speed)`.
- `length_squared()`: calcula $x^2 + y^2$ sin el costo de calcular la raíz cuadrada.
- `normalized()`: divide cada componente entre su longitud solo si esta es mayor a `0.0001f`, evitando divisiones entre cero cuando el personaje está quieto.

5. Modularización: mover `Vector2` a su propia cabecera `Vector2.hpp`

Tener la estructura en `main.cpp` contamina el archivo principal. Ahora la extraeremos a su propio archivo de cabecera.

1. Dentro de la carpeta `src/`, crea un nuevo archivo llamado `Vector2.hpp`.

2. Escribe en `src/Vector2.hpp` el código de la estructura con la protección `#pragma once`:

```
#pragma once

#include <cmath>

struct Vector2
{
    float x{0.0f};
    float y{0.0f};

    constexpr Vector2() = default;
    constexpr Vector2(float x, float y) : x(x), y(y) {}

    Vector2 operator+(const Vector2 &other) const
    {
        return {x + other.x, y + other.y};
    }

    Vector2 operator-(const Vector2 &other) const
    {
        return {x - other.x, y - other.y};
    }

    Vector2 operator*(float scalar) const
    {
        return {x * scalar, y * scalar};
    }

    float length_squared() const
    {
        return x * x + y * y;
    }

    float length() const
    {
        return std::sqrt(length_squared());
    }
}
```

```

Vector2 normalized() const
{
    float len = length();
    if (len > 0.0001f)
    {
        return {x / len, y / len};
    }
    return {0.0f, 0.0f};
}
};

```

3. Vuelve a `src/main.cpp`, **borra** la definición de `struct Vector2` que pusiste en el paso 4, y en la zona superior de inclusiones agrega:

```
#include "Vector2.hpp"
```

Compila nuevamente con:

```
cmake --build build
```

Verifica que compila limpiamente. Como `Vector2.hpp` es un encabezado incluido por `main.cpp`, no hace falta modificar el `CMakeLists.txt`.

6. Movimiento vectorial y corrección del *bug* diagonal

Ahora integraremos `Vector2` en el ciclo de movimiento para normalizar la dirección.

En `src/main.cpp`, modifica `AppState` para reemplazar las variables sueltas `rect_x` y `rect_y` por un vector:

```

struct AppState
{
    SDL_Renderer *renderer{nullptr};
    SDL_Window *window{nullptr};

    Uint64 last_ticks{0};

    // Usamos Vector2 para la posición

```

```

    Vector2 player_pos{440.0f, 240.0f};
    float rect_width{60.0f};
    float rect_height{60.0f};
    float speed{300.0f};
} appstate;

```

Ahora, en `SDL_AppIterate`, reemplaza el bloque de actualización del paso 2 por la lógica vectorial:

```

// 2. Fase de Actualización (Update)
const bool *keys = SDL_GetKeyboardState(nullptr);

Vector2 input_dir{0.0f, 0.0f};

if (keys[SDL_SCANCODE_W] || keys[SDL_SCANCODE_UP])    input_dir.y -=
1.0f;
if (keys[SDL_SCANCODE_S] || keys[SDL_SCANCODE_DOWN]) input_dir.y +=
1.0f;
if (keys[SDL_SCANCODE_A] || keys[SDL_SCANCODE_LEFT]) input_dir.x -=
1.0f;
if (keys[SDL_SCANCODE_D] || keys[SDL_SCANCODE_RIGHT]) input_dir.x +=
1.0f;

// Normalización: si el jugador se está moviendo, normalizamos el vector
if (input_dir.length_squared() > 0.0f)
{
    input_dir = input_dir.normalized();
}

// Calculamos el desplazamiento: velocidad escalar * dirección unitaria
* delta_time
Vector2 displacement = input_dir * (app->speed * delta_time);
app->player_pos = app->player_pos + displacement;

```

Finalmente, en la fase de renderizado de `SDL_AppIterate`, actualiza el `SDL_FRect` con las coordenadas del vector:

```

// 3. Fase de Renderizado
SDL_SetRenderDrawColor(app->renderer, 30, 30, 35, 255);
SDL_RenderClear(app->renderer);

```

```
SDL_SetRenderDrawColor(app->renderer, 60, 180, 100, 255);
SDL_FRect player_rect{app->player_pos.x, app->player_pos.y, app->rect_width, app->rect_height};
SDL_RenderFillRect(app->renderer, &player_rect);

SDL_RenderPresent(app->renderer);
```

Compila y ejecuta:

```
cmake --build build
./build/practica02_motor
```

Prueba moverte en diagonal: notarás que la velocidad en diagonal ahora es exactamente idéntica al movimiento horizontal o vertical. ¡El *bug* está solucionado!

7. Hacia la arquitectura del motor: `struct Character` (Composición)

En lugar de tener la posición, el tamaño y la velocidad flotando dispersos en `AppState`, agrupémoslos en una entidad con significado conceptual.

En `src/main.cpp`, define justo arriba de `AppState` la siguiente estructura:

```
struct Character
{
    Vector2 position{440.0f, 240.0f};
    Vector2 size{60.0f, 60.0f};
    float speed{300.0f};
    SDL_Color color{60, 180, 100, 255};
};
```

Observa la **composición**: `Character` *contiene* una variable `position` y una variable `size` de tipo `Vector2`.

Ahora actualiza `AppState` para que contenga a este personaje:

```
struct AppState
{
    SDL_Renderer *renderer{nullptr};
    SDL_Window *window{nullptr};
    Uint64 last_ticks{0};

    Character player;
} appstate;
```

Actualiza en `SDL_AppIterate` las referencias para usar `app->player`:

```
// Desplazamiento
Vector2 displacement = input_dir * (app->player.speed * delta_time);
app->player.position = app->player.position + displacement;

// Renderizado
SDL_SetRenderDrawColor(app->renderer, app->player.color.r, app->player.color.g, app->player.color.b, app->player.color.a);
SDL_FRect player_rect{app->player.position.x, app->player.position.y, app->player.size.x, app->player.size.y};
SDL_RenderFillRect(app->renderer, &player_rect);
```

Recompila y verifica que el código funciona exactamente igual, pero con una organización arquitectónica mucho más limpia y escalable.

8. Desacoplamiento de Físicas: Timestep Fijo (*Fixed Timestep*)

Para experimentar cómo los motores separan la actualización de físicas del renderizado, agregaremos un acumulador de tiempo a `AppState`:

```
struct AppState
{
    SDL_Renderer *renderer{nullptr};
    SDL_Window *window{nullptr};
    Uint64 last_ticks{0};

    Character player;
```

```
// Acumulador de tiempo para físicas
float physics_accumulator{0.0f};
} appstate;
```

Crea una función para la actualización física a paso constante:

```
void PhysicsUpdate(Character &character, const Vector2 &direction, float
fixed_dt)
{
    Vector2 displacement = direction * (character.speed * fixed_dt);
    character.position = character.position + displacement;
}
```

En `SDL_AppIterate`, en lugar de mover el personaje directamente con `delta_time`, procesa las físicas en bloques exactos de $1/60 \approx 0.01666$ s:

```
constexpr float FIXED_TIMESTEP = 1.0f / 60.0f; // 60 Hz estables
app->physics_accumulator += delta_time;

while (app->physics_accumulator >= FIXED_TIMESTEP)
{
    PhysicsUpdate(app->player, input_dir, FIXED_TIMESTEP);
    app->physics_accumulator -= FIXED_TIMESTEP;
}
```

Con este patrón, incluso si el renderizado fluctúa o cae a 20 FPS, la simulación física avanzará en pasos discretos idénticos y predecibles (bueno si es que no colapsó toda tu computadora).

9. (Avanzado, opcional) Confinamiento o persecución del mouse

Elige e implementa al menos uno de los siguientes retos adicionales:

Opción A: Confinar al personaje en la pantalla (*Clamping*)

Evita que el rectángulo salga de los límites de la ventana (960×540).

- El límite izquierdo es $x \geq 0$.
- El límite derecho es $x \leq 960 - \text{size.x}$.

- El límite superior es $y \geq 0$.
- El límite inferior es $y \leq 540 - \text{size.y}$.

Opción B: Persecución del cursor del mouse

Utiliza `SDL_GetMouseState(&mouse_x, &mouse_y)` para obtener la posición actual del cursor. Calcula el vector que apunta desde el personaje hacia el mouse:

$$\vec{d} = \vec{pos}_{\text{mouse}} - \vec{pos}_{\text{personaje}}$$

Normaliza ese vector y haz que el personaje se mueva constantemente hacia el cursor solo si la distancia es mayor a un radio de tolerancia (para evitar que vibre al llegar).

10. Publicación del proyecto en GitHub

A partir de esta práctica, tus entregas se gestionan mediante repositorios en GitHub.

1. Inicia sesión en tu cuenta de [GitHub](#).
2. Crea un nuevo repositorio llamado `practica02-motor` (o el nombre que prefieras). Selecciónalo como **Público** y **no** marques las casillas de agregar README ni `.gitignore` (ya tenemos los nuestros).
3. En tu computadora, dentro de la carpeta de la práctica (donde están `CMakeLists.txt` y `src/`), inicializa git si aún no lo has hecho:

```
git init
```

4. Agrega los archivos y haz tu primer commit:

```
git add .
git commit -m "Práctica 02: Implementación de Vector2, normalización, CMake y composición"
```

Commits

Si durante el desarrollo encuentras puntos donde *algo* funciona, y quieres guardarlos antes de avanzar a algo que podría romperlos; ese es un muy buen momento para crear un commit. Un commit es un *checkpoint* donde guardas tu

partida, y te permite regresar si algo va mal despues. Así te evitas tener archivo, archivo_final, archivo_final_final, archivo_final_final2, etc.

5. Renombra la rama a `main`, enlaza tu repositorio remoto de GitHub y sube tu código:

```
# Cambiar el nombre de la rama a main
git branch -M main

# Vincular con tu repositorio de GitHub (reemplaza 'tu-usuario' con tu
usuario real)
git remote add origin https://github.com/tu-usuario/practica02-motor.git

# Subir tu historial al repositorio remoto
git push -u origin main
```

Entra en tu navegador a tu repositorio de GitHub y comprueba que tus archivos `CMakeLists.txt`, `src/main.cpp` y `src/Vector2.hpp` están visibles en la nube.

Entrega

Para evaluar la práctica, deberás entregar en Google Classroom:

1. **El enlace a tu repositorio público de GitHub**, por ejemplo: `https://github.com/tu-usuario/practica02-motor`.
2. **El binario compilado** `practica02_motor` (o `practica02_motor.exe` en Windows).
3. **Un reporte breve (1 a 2 páginas en PDF o Markdown)** que contenga:
 - Nombre completo y usuario de GitHub.
 - Demostración y cálculo matemático del error diagonal: explica con tus propias palabras por qué ocurría y cómo la normalización vectorial lo corrigió.
 - Captura de pantalla de tu ventana ejecutándose con el personaje en movimiento.
 - Captura o enlace a tu commit en GitHub.
 - Si realizaste el ejercicio opcional (límites de pantalla o persecución de mouse), incluye una captura y explica brevemente cómo lo implementaste.

Subida a Classroom

1. Sube tu reporte en PDF junto con el binario `practica02_motor`.

2. En el campo de **comentario privado** de la entrega (o en el texto del reporte), escribe la URL de tu repositorio de GitHub.
-

Verificación

Antes de entregar, verifica que cumplas con todos los puntos:

- ☐ El proyecto compila limpiamente utilizando `cmake -B build && cmake --build build`.
- ☐ La estructura `Vector2` está modularizada en su propio archivo de cabecera `src/Vector2.hpp`.
- ☐ `Vector2.hpp` cuenta con la directiva `#pragma once` e implementa suma, resta, escala, longitud y normalización con protección contra división entre cero.
- ☐ El objeto se desplaza suavemente usando las teclas WASD o flechas de dirección.
- ☐ Al moverse en diagonal, la velocidad es uniforme respecto al movimiento en línea recta gracias a la normalización.
- ☐ Los datos del jugador están agrupados dentro de un `struct Character` (composición).
- ☐ El código está publicado en un repositorio de GitHub accesible mediante tu enlace.
- ☐ El reporte incluye la explicación matemática del *bug* diagonal y las capturas solicitadas.